Ejercicio 6 · Polimorfismo vía Interfaces

1. Requisitos y decisiones de análisis

Requisito del enunciado	Decisión implementada
Múltiples APIs y ampliación futura	10 objetos iniciales: 4 pagos, 4 logística y 2 FEL; nuevos proveedores de categorías existentes se agregan al catálogo sin alterar el controlador ni el motor.
Contrato financiero, documentos y consultas	Interfaces Transaccional, Documentable y Consultable; Auditable para el registro de eventos.
Reintegro logístico incompatible	Despachador Visitor devuelve Resultado(NO_SOPORTADO), sin excepción y sin colapsar la compra.
Catálogo, búsqueda y latencia	Una sola List<IntegracionApi>; búsqueda por ID; Collections.sort mediante Comparable<IntegracionApi>.
Encapsulamiento y buenas prácticas	Campos privados, getters, mutador de estado validado, equals/hashCode por ID, toString, listas expuestas como no modificables.
MVC e interfaz gráfica	Modelo + servicios, ControladorIntegraciones y vistas independientes de consola / Swing.
Overloading	MotorCheckout.cobrar(Transaccional, BigDecimal, String) y cobrar(Transaccional, String, String); consultar con y sin referencia.

2. Supuestos verificables

Se simulan las respuestas de proveedores, latencias y endpoints; no hay comunicación HTTP ni procesamiento bancario real. La latencia es un atributo promedio configurado, no una medición de red. Para probar validaciones se incluye LOG-03 desconectada, PAG-03 degradada y el resto conectadas. La configuración no almacena secretos. La auditoría es temporal, en memoria.

3. Contratos, responsabilidades y relaciones

Elemento	Responsabilidad / contrato
IntegracionApi (abstracta)	Mantiene identidad, proveedor, endpoint, latencia, estado y auditoría. Implementa Comparable y Auditable; declara aceptar(OperacionIntegracion) para doble despacho.
Transaccional	cobrar(monto, referencia), reintegrar(monto, referencia): Resultado. Exclusiva de PasarelaPago.
Documentable	generarDocumento(referencia): Resultado. Implementada por ProveedorLogistico y CertificadorFel.
Consultable	consultarEstado(referencia): Resultado. Implementada por las tres categorías.
PasarelaPago	Encapsula cobros, reintegros y consulta de pagos; valida monto, referencia y conexión.
ProveedorLogistico	Encapsula rastreo y generación de guías; no implementa Transaccional.
CertificadorFel	Encapsula consulta y generación de XML FEL simulado; no implementa Transaccional.
CatalogoIntegraciones	Único dueño de List<IntegracionApi>; valida IDs, busca, lista y ordena.
MotorCheckout	Consume contratos por parámetros de interfaz: Transaccional, Documentable y Consultable.
DespachadorOperaciones	Visitor de operaciones; selecciona comportamiento permitido por tipo sin instanceof en la lógica central.
ControladorIntegraciones	Coordina búsqueda, listado, ordenamiento, auditoría y validaciones del menú.
VistaConsola / VistaGrafica	Capturan y presentan datos sin acceder directamente al catálogo ni implementar reglas del dominio.

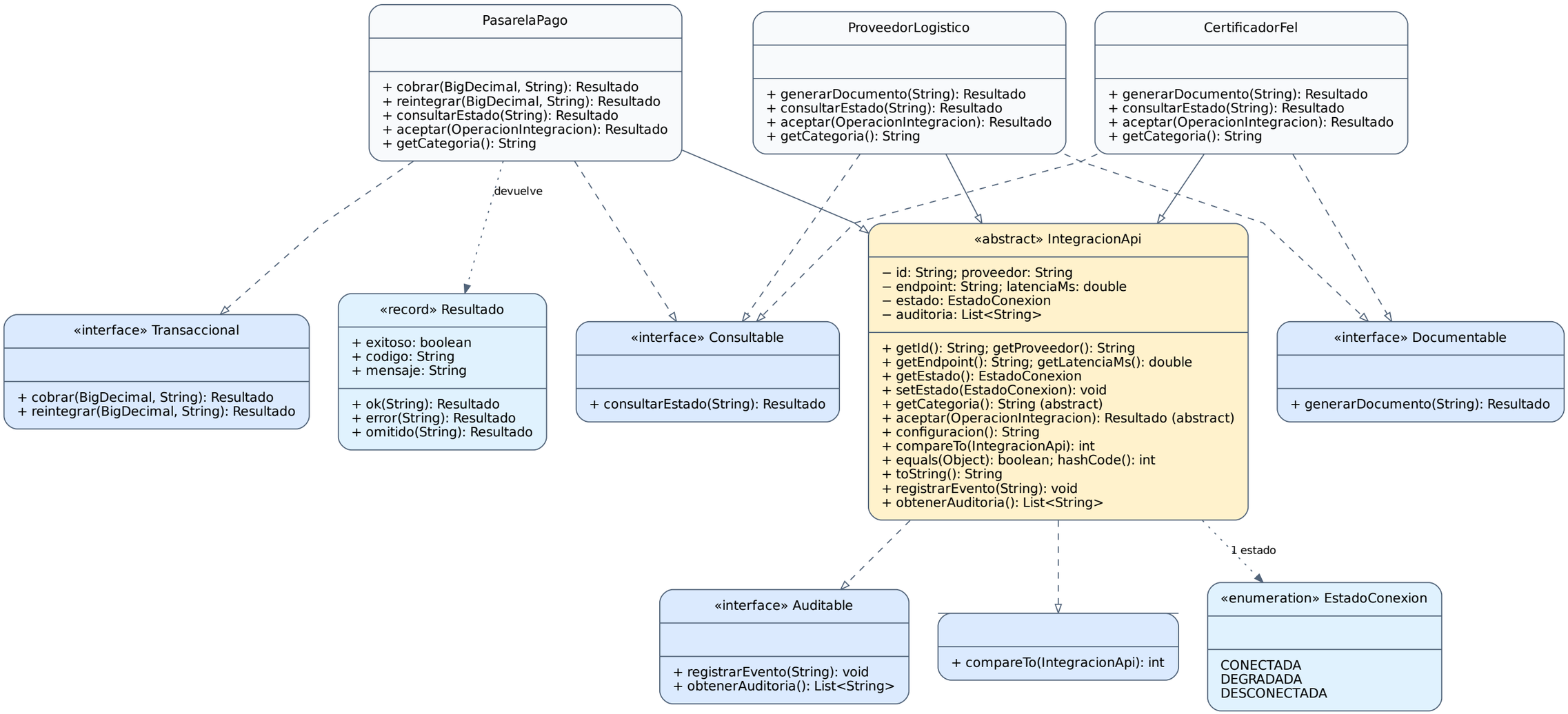
4. Aplicación de conceptos de POO

Herencia: las tres clases concretas extienden IntegracionApi. Polimorfismo: la lista guarda IntegracionApi y cada objeto ejecuta su propio aceptar(). Interfaces como tipos de parámetros: MotorCheckout recibe Transaccional, Documentable o Consultable, no clases concretas. Sobreloading: cobrar() admite monto BigDecimal o String. Overriding: clases concretas sobrescriben aceptar(), getCategoria() y métodos contractuales. Comparable: compara latencia y desempata por ID. equals/hashCode: compara IDs sin distinguir mayúsculas.

Nota sobre instanceof: solo se utiliza en equals(Object) para comprobar el tipo del argumento de Object, práctica estándar de igualdad; el flujo de negocio, resumen de integraciones y selección de operaciones no utilizan instanceof ni conversiones explícitas.

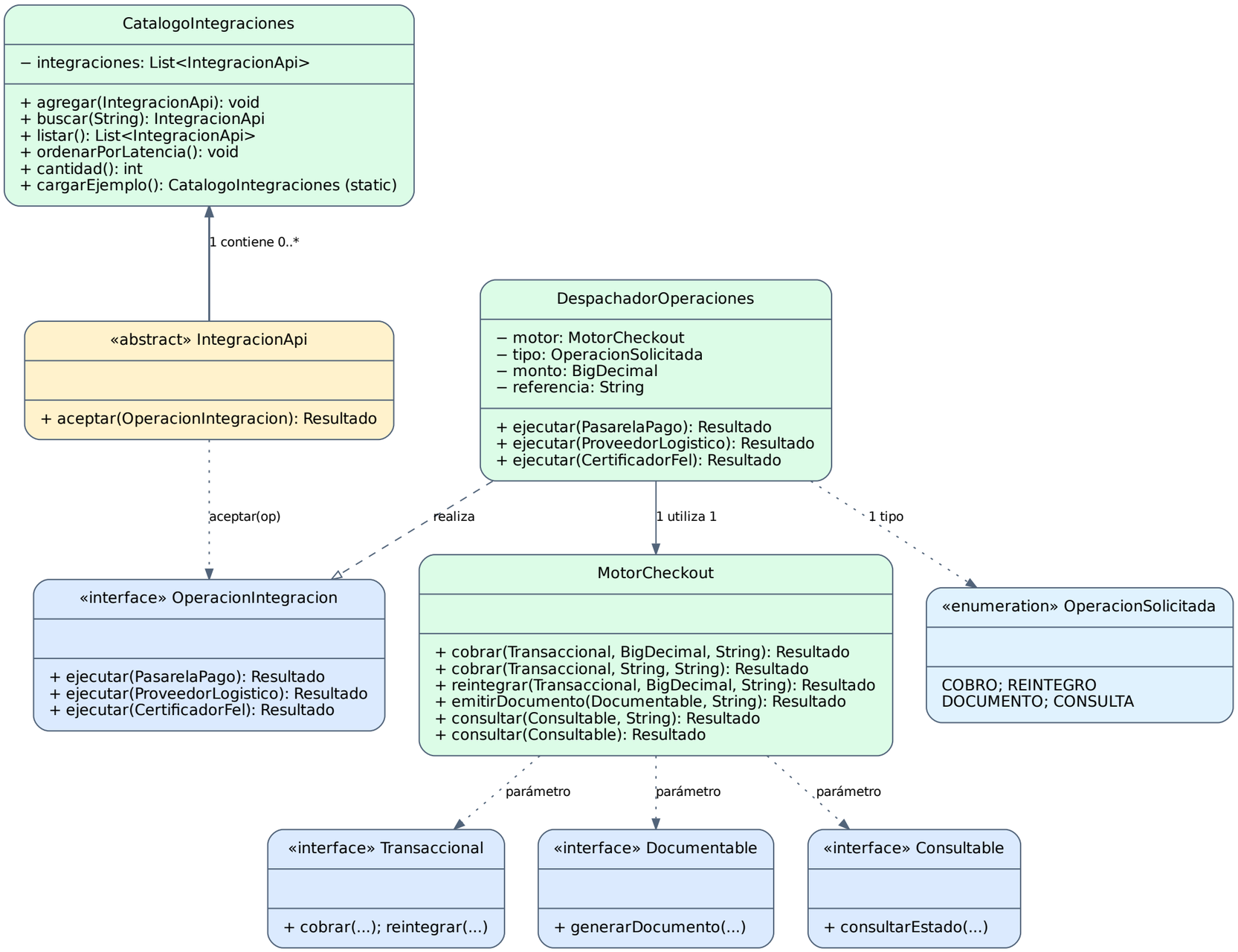
UML 1 · Modelo, herencia e interfaces

Las flechas con triángulo vacío continuo indican herencia; las discontinuas indican realización de interfaz. Se muestran atributos, firmas y visibilidades.



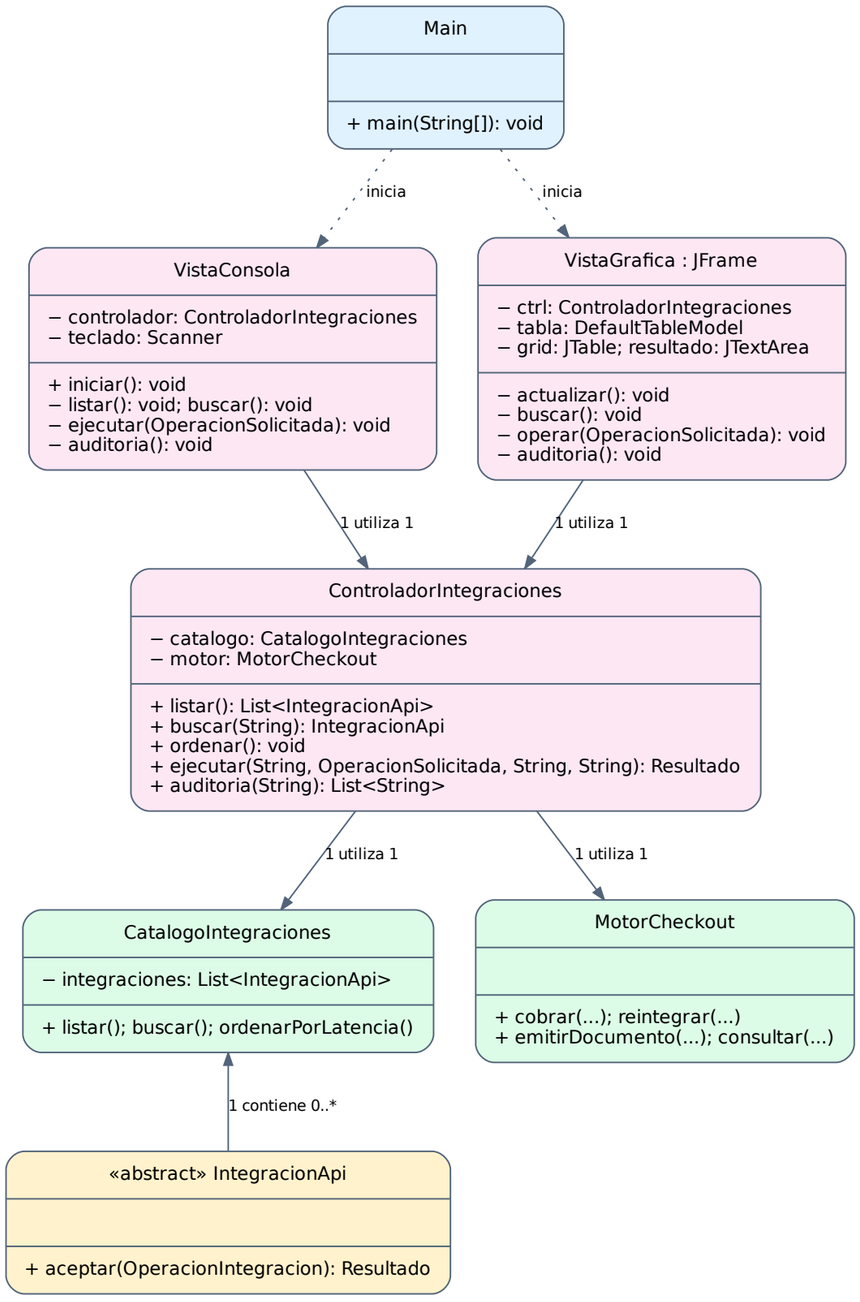
UML 2 · Servicios, catálogo y despacho polimórfico

La única colección de integraciones vive en CatalogoIntegraciones. MotorCheckout consume interfaces como parámetros y el Visitor resuelve las operaciones compatibles.



UML 3 · Patrón MVC y relaciones de dependencia

Las vistas de consola y Swing dependen del controlador. El controlador coordina catálogo y motor, sin mezclar responsabilidades de presentación y dominio.



6. Flujos de operación y manejo de estados

Carga: CatalogoIntegraciones.cargarEjemplo() crea diez conexiones simuladas con IDs únicos y sus estados. Listado: devuelve solo una vista inmodificable de la lista de integraciones y presenta también 0 eventos cuando aplica. Búsqueda: acepta ID en cualquier combinación de mayúsculas/minúsculas, y si no existe se informa el caso. Ordenamiento: sort sobre Comparable ascendente por latencia, con desempate lexicográfico por ID.

Cobro: Vista → Controlador (validación de monto positivo, máximo 2 decimales y referencia) → IntegracionApi.aceptar(Visitor) → Visitor.ejecutar(PasarelaPago) → MotorCheckout.cobrar(Transaccional, ...) → PasarelaPago. Reintegro no aplicable: el mismo flujo termina en Visitor.ejecutar(ProveedorLogistico) o ejecutar(CertificadorFel), que devuelve NO_SOPORTADO. En ningún caso se emite una excepción por incompatibilidad.

Documentos: la interfaz Documentable solo se usa para guías y FEL; las pasarelas responden NO_SOPORTADO. Consulta: las tres categorías usan Consultable. Desconexión: si una operación soportada llega a una API DESCONECTADA, devuelve ERROR explícito. Estado degradado: la operación sigue permitida en este prototipo y su situación se muestra en el catálogo.

7. Matriz de capacidades

Categoría	Cobro	Reintegro	Documento	Consulta	Auditoría
PasarelaPago	Sí	Sí	No soportado	Sí	Sí
ProveedorLogistico	No soportado	No soportado	Guía	Rastreo	Sí
CertificadorFel	No soportado	No soportado	XML simulado	Estado FEL	Sí

8. Extensibilidad y decisión de arquitectura

Los proveedores nuevos de las categorías actuales se crean como instancias adicionales, sin modificar la lógica del checkout. La única lista de integraciones sigue siendo List<IntegracionApi>. El Visitor permite no usar instanceof en la lógica de negocio, pero tiene una compensación: para incorporar una cuarta categoría completamente nueva sí sería necesario ampliar OperacionIntegracion y DespachadorOperaciones. Esta limitación no se oculta y es diferente de añadir un nuevo proveedor logístico, que ya está soportado.

9. Evidencia de pruebas (47 verificaciones automáticas)

Área	Casos ejecutados
Carga y catálogo	10 APIs; distribución 4/4/2; estado desconectado; configuración; búsqueda existente/no existente; latencia ordenada; mínimo primero.
Transacciones	Cobro válido; reintegro válido; reintegro de logística omitido; cobro de FEL omitido; documento en pasarela omitido.
Documentos y consultas	Guía logística; XML FEL simulado; rastreo; consulta de pasarela degradada; error de consulta desconectada.
Validaciones	Montos negativos, cero, más de dos decimales, texto no numérico; referencia vacía; API inexistente.
Buenas prácticas	Auditoría con eventos y vacía; equals/hashCode; colección inmodificable; ID duplicado rechazado; startup adicional (11).
Sobrecarga	Cobro por String; cobro por BigDecimal; consulta sin referencia.
Menú (todas las opciones)	Opción inválida; búsqueda ausente; ordenar; cobrar; reintegrar incompatible; generar FEL; consultar; auditoría con cero; salir; conteo de 11.

10. Reproducción de pruebas y entrega

Compilación: **javac -encoding UTF-8 -d out [archivos .java]**. Ejecución consola: **java -cp out tienda.Main**. Interfaz gráfica: **java -cp out tienda.Main --gui**. Pruebas: **java -cp out tienda.pruebas.Pruebas**. Se proporcionan scripts Windows/Linux y README detallado.

Resultado de prueba local: 47 verificaciones correctas con Java 21. Se incluye archivo de salida de pruebas. El diseño UML representa los componentes principales y sus firmas públicas relevantes; los archivos fuente Java constituyen la especificación íntegra del prototipo.

11. Trazabilidad con la rúbrica y correcciones anteriores

La documentación permite verificar explícitamente la lista polimórfica, interfaces, parámetros por interfaz, overloading, MVC, buenas prácticas y Comparable. Se corrigió el patrón de exponer colecciones internas, se evita el uso de instanceof en decisiones funcionales, no se omiten ceros en reportes, se valida cada opción del menú y se documentan pruebas reproducibles. No se conservan referencias de asignaciones a jornadas porque ese era un error de otro ejercicio y no forma parte de los requisitos del Ejercicio 6.

12. Auditoría complementaria de calidad

Ampliación posterior a la primera revisión: 18 pruebas adicionales.

Cobertura total: 47 pruebas base + 18 pruebas adicionales = 65 verificaciones.

Casos adicionales verificados

- Cobros: monto mínimo positivo, monto alto, monto en blanco.
- Reintegros: mínimo, más de dos decimales, monto no numérico.
- Documentos: rechazo si la API está desconectada y referencia vacía.
- FEL: generación correcta en el segundo certificador.
- Operaciones incompatibles: cobro en logística y reintegro en FEL.
- Validaciones: operación nula y referencia nula.
- Estados: desconexión bloquea cobro; reconexión permite continuar.
- Encapsulamiento: auditoría no modificable desde fuera.
- Comparable: orden ascendente con desempate lexicográfico por ID.
- Búsqueda: identificador con espacios alrededor.

Evidencias ejecutables

```
java -cp out tienda.pruebas.Pruebas
java -cp out tienda.pruebas.PruebasAdicionales
```

Archivos: Resultados_pruebas.txt y Resultados_pruebas_adicionales.txt

Alcance de la revisión

Código compilado con JDK 21; 65/65 pruebas automáticas correctas.

UML 1, 2 y 3 completos, sin recortes, con salida vectorial.

Las operaciones de proveedores externos continúan siendo simuladas.

La interfaz Swing requiere comprobación manual en un escritorio.

La calificación final corresponde al docente, no a la suite de pruebas.